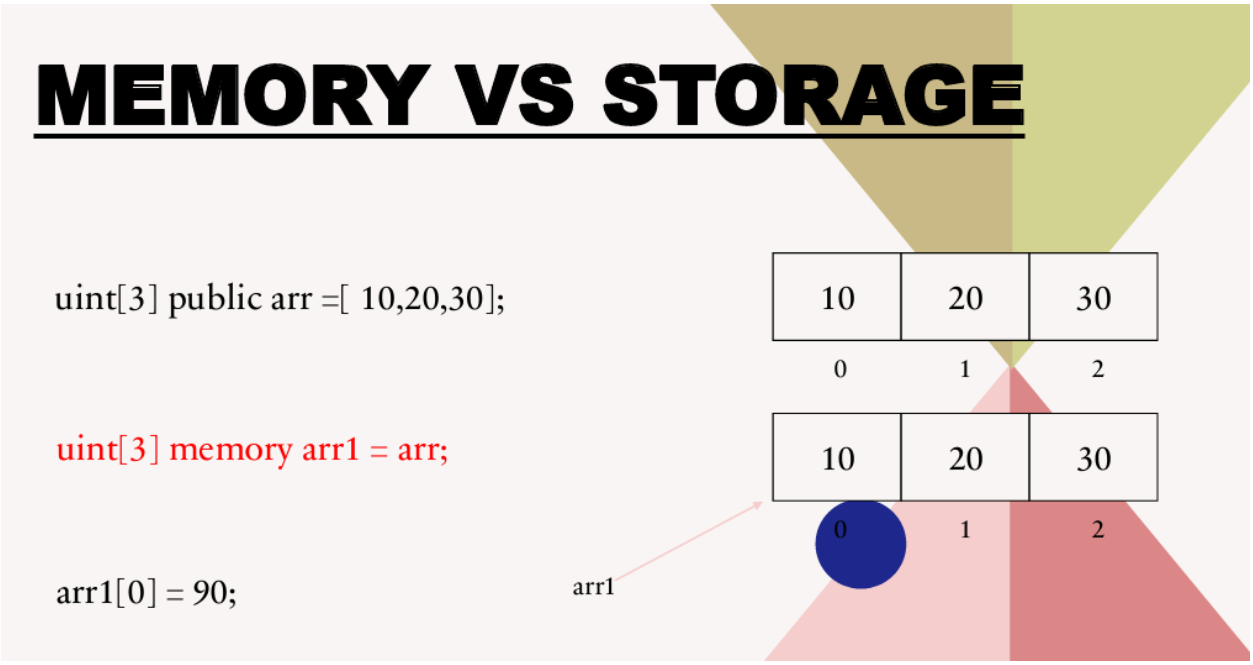
# MEMORY VS CALLDATA

| Memory | Calldata |
| --- | --- |
| Can be used anywhere inside a function | Can be used only in a function argument |
| Mutable | Immutable |

```
// ✅ Example of calldata in Solidity

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract CallDataExample {
    function getFirstCharacter(string calldata name)
        external
        pure
        returns (bytes1)
    {
        return bytes(name)[0]; // Reads but does not modify calldata
    }
}

// Function Parameter (string calldata name)

// The name variable is stored in calldata, meaning it cannot be modified.
// This is gas-efficient because calldata avoids unnecessary copying.

contract MemoryVsCalldata {
    // ✅ Function using 'memory' (can modify string)
    function modifyStringMemory(string memory text)
        public
        pure
```

```
        returns (string memory)
    {
        text = "Modified String"; // ✅ Allowed: memory variables can be
changed
        return text;
    }

    // ❌ Function using 'calldata' (cannot modify string)
    function readOnlyStringCalldata(string calldata text)
        external
        pure
        returns (string calldata)
    {
        // text = "Modified"; ❌ Not Allowed: calldata is immutable
        return text;
    }
}
```

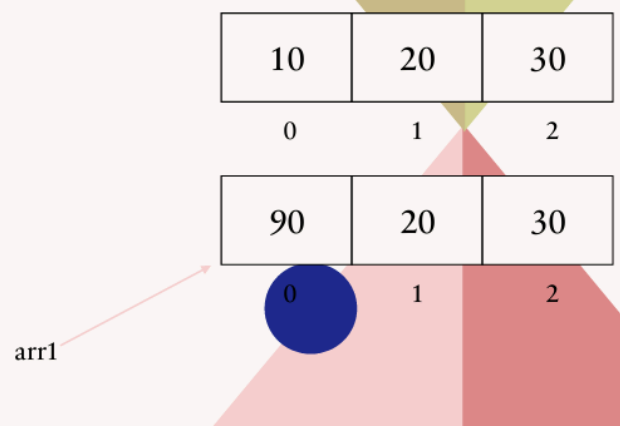


# MEMORY VS STORAGE

uint[3] public arr =[ 10,20,30];

uint[3] memory arr1 = arr;

arr1[0] = 90;

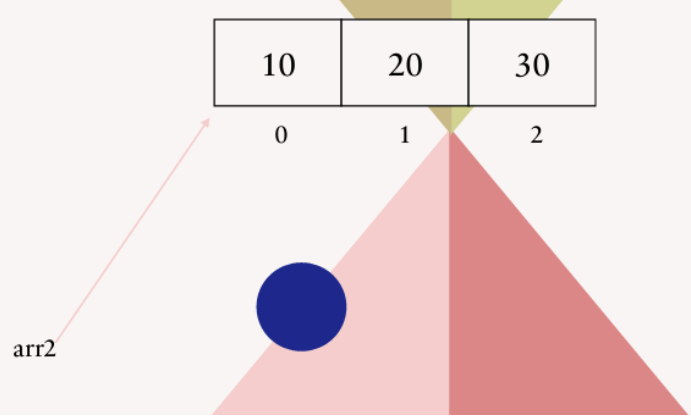


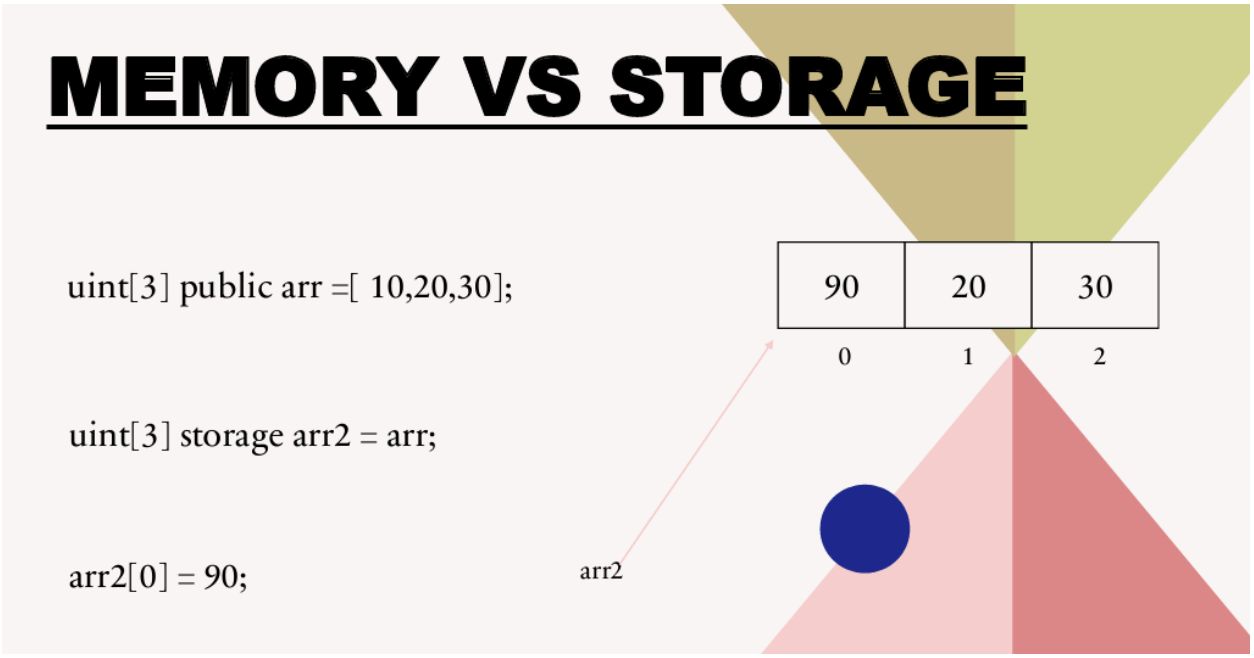

# MEMORY VS STORAGE

uint[3] public arr =[10,20,30];

uint[3] storage arr2 = arr;

arr2[0] = 90;






## MEMORY VS STORAGE

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract MyContract {
    uint[3] public arr = [10, 20, 30];

    // Function using storage (modifies the state variable directly)
    function storageArr() external returns (uint[3] memory) {
        uint[3] storage stoArr = arr;
        stoArr[0] = 1; // Changes reflect in 'arr'
        return arr; // Returning 'arr' instead of 'stoArr' (storage
variables can't be returned)
    }

    // Function using memory (creates a copy and doesn't modify the
original state)
    function memoryArr() external view returns (uint[3] memory) {
        uint[3] memory memArr = arr;
        memArr[0] = 1; // Changes do NOT reflect in 'arr'
        return memArr;
    }
}
```

# MAPPING VS ARRAY

| _roll | | name |
|---|---|---|
| 1 | → | Ravi |
| 2 | → | John |
| 4 | → | Alice |
| 7 | → | Akash |

Mapping

# MAPPING VS ARRAY

| index | name |
|---|---|
| 0 | |
| 1 | Ravi |
| 2 | John |
| 3 | |
| 4 | Alice |
| 5 | |
| 6 | |
| 7 | Akash |

Array

# MAPPING VS ARRAY

| index | name |
|---|---|
| 0 | |
| 1 | Ravi |
| 2 | John |
| 3 | |
| 4 | Alice |
| 5 | |
| 6 | |
| 7 | Akash |

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

// Creating a Library
library MathLibrary {
    function add(uint a, uint b) internal pure returns (uint) {
        return a + b;
    }

    function multiply(uint a, uint b) internal pure returns (uint) {
        return a * b;
    }
}

// Using Library in a Contract
contract Calculator {
    // using MathLibrary for uint;  // Attaching the library to uint type

    function getSum(uint x, uint y) public pure returns (uint) {
        return MathLibrary.add(x,y); // Uses MathLibrary's add function
    }
```

```
    // function getProduct(uint x, uint y) public pure returns (uint) {
    //     return x.multiply(y); // Uses MathLibrary's multiply function
    // }
}
```

# EXTERNAL LIBRARY

- An external library in Solidity is a special type of contract deployed on the blockchain that other contracts can use to execute code without including the library's code in their own bytecode.

External libraries have the following characteristics:

1. **Delegate Call Usage**: External libraries can only be invoked using delegatecall, which executes the library's code in the context of the calling contract. This means that the library's code operates on the state variables of the calling contract, not its own.

2. **Stateless**: Libraries themselves do not have state. They are purely for code reuse and do not have their own storage. Any state changes made by the library functions affect the storage of the calling contract.

3. **Deployed Once**: Libraries are deployed separately and their addresses are used by contracts that wish to use them. This means the same library can be reused by many different contracts.

While external libraries can technically define state variables, it is not a practical or safe design pattern in Solidity. External libraries should be used for stateless utility functions, ensuring they remain reusable and secure.

# EXTERNAL LIBRARY

```
library MathsLibrary{

    function add(uint a,uint b) external pure returns(uint){
        return a+b;
    }
     function mul(uint a,uint b) external pure returns(uint){
        return a*b;
    }
    function sub(uint a,uint b) external pure returns(uint){
        return a-b;
    } function div(uint a,uint b) external pure returns(uint){
        return a/b;
    }
}
```

```
contract MyContract {
    // Address of the deployed MathLibrary
    address constant MATH_LIBRARY_ADDRESS = 0xf8e81D47203A594245E36C48e151709F0C19fBe8;

    function useLibraryFunction(uint256 a, uint256 b) public view returns (uint256) {
        (bool success, bytes memory returnData) = MATH_LIBRARY_ADDRESS.staticcall(
            abi.encodeWithSignature("multiply(uint256,uint256)", a, b)
        );
        require(success, "Library call failed");

        return abi.decode(returnData, (uint256));
    }
}
```

# INTERNAL VS EXTERNAL LIBRARY

| Feature | External Library | Internal Library |
|---|---|---|
| **Deployment** | Deployed as a separate contract | Part of the contract that uses them |
| **Gas Cost** | Higher due to external calls | Lower as they are internal function calls |
| **Usage** | Called via DELEGATECALL | Directly linked during contract compilation |
| **Security** | More susceptible to security issues due to external calls | Less susceptible due to internal linkage |
| **Reusability** | Can be reused across multiple contracts | Reused only within the contract or contracts that include it |